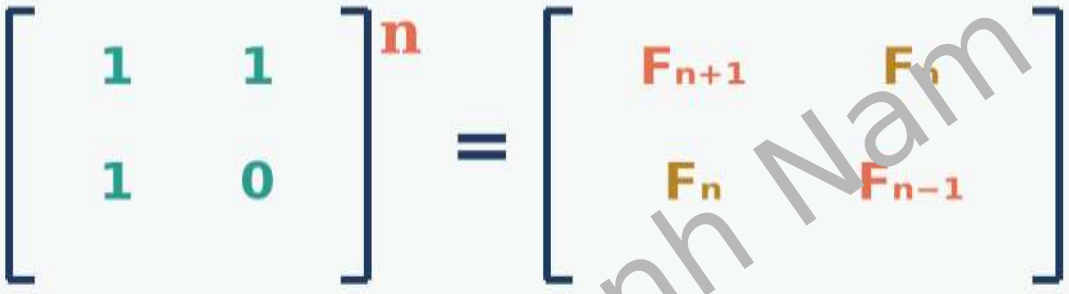
TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

Tập 8 · 16 bài Codeforces, dịch tiếng Việt

MA TRẬN VÀ DÃY TRUY HỒI



Fibonacci thứ 10^{18} chỉ cần khoảng 120 phép nhân ma trận 2 × 2

$f_x = c^{2x-6} \cdot f_{x-1} \cdot f_{x-2} \cdot f_{x-3}$ → số mũ theo modulo M − 1

Lê Thành Nam

Nhân ma trận | Lũy thừa ma trận | Truy hồi tuyến tính | Số mũ mod M − 1

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

LÊ THÀNH NAM

Tổng hợp và biên soạn

MỤC LỤC

Ngây thơ → Bế tắc → Phá bế tắc → Tối ưu

Giới thiệu 2

Chương 1. Truy hồi tuyến tính bậc thấp 3

Bài 1. Jzzhu và các dãy số *(Codeforces 450B)* 4
Bài 2. Cây trồng *(Codeforces 185A)* 7
Bài 3. Tứ diện *(Codeforces 166E)* 10

Chương 2. Lũy thừa ma trận 13

Bài 4. Hàm tuyến tính lặp *(Codeforces 678D)* 14
Bài 5. Đá quý thần kì *(Codeforces 1117D)* 17
Bài 6. Giải mã bộ gen *(Codeforces 222E)* 21
Bài 7. Cá mập và các khối số *(Codeforces 621E)* 25
Bài 8. Dãy XOR *(Codeforces 691E)* 29

Chương 3. Ma trận cho các truy hồi phức tạp 33

Bài 9. Lại một dãy số nữa *(Codeforces 392C)* 34
Bài 10. GukiZ và các phép toán nhị phân *(Codeforces 551D)* 38
Bài 11. Sasha và mảng *(Codeforces 718C)* 42
Bài 12. Darth Vader và cây *(Codeforces 514E)* 47
Bài 13. Bài toán người chạy *(Codeforces 954F)* 51
Bài 14. Okabe và El Psy Kongroo *(Codeforces 821E)* 55

Chương 4. Số mũ theo modulo M – 1 59

Bài 15. Dãy truy hồi dạng tích *(Codeforces 1182E)* 60
Bài 16. Năm mới và dãy truy hồi *(Codeforces 1106F)* 64

Lê Thành Nam

GIỚI THIỆU

Tài liệu này dành cho ai

Đây là Tập 8 của bộ Tuyển tập bài tập Toán trên Codeforces, gồm 16 bài về ma trận và dãy truy hồi, sắp thành 4 chương: *truy hồi tuyến tính bậc thấp; lũy thừa ma trận; ma trận cho các truy hồi phức tạp; số mũ theo modulo $M - 1$*. Các bài tiêu biểu: 450B Jzzhu and Sequences; 185A Plant; 166E Tetrahedron; 222E Decoding Genome; 1182E Product Oriented Recurrence. Hai bài 1182E và 1106F đã xuất hiện ở Tập 3; ở đây chúng được trình bày lại với trọng tâm là ma trận số mũ. Tài liệu phù hợp với học sinh đã học Tập 3 (số học modulo), đang luyện thi học sinh giỏi cấp tỉnh, cấp quốc gia và các kì thi tương đương.

Mỗi bài gồm hai phần. Đề bài được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc, định dạng vào/ra và ví dụ của đề gốc, kèm bảng subtask do tài liệu bổ sung. Hướng dẫn giải đi qua 4 bước tư duy; mỗi bước có code chạy được, có hình minh họa ma trận và trạng thái, bài nào cũng kết thúc bằng bảng so sánh các tầng lời giải đo thật trên máy.

4 bước tư duy giải một bài

Bước	Nội dung
Ngây thơ	Ý tưởng tự nhiên nhất, làm đúng như đề mô tả; có code và độ phức tạp.
Bế tắc	Biểu đồ thời gian đo thật, chỉ ra chỗ chậm hoặc chỗ sai, kết thúc bằng một câu hỏi then chốt.
Phá bế tắc	Nhận xét hoặc tính chất giúp vượt qua chỗ tắc, có ví dụ và hình minh họa.
Tối ưu	Code cuối cùng, độ phức tạp và các lưu ý: kiểu dữ liệu, tràn số, đọc ghi nhanh.

Lê Thành Nam

Subtask và bảng so sánh các tầng

Đề Codeforces không chia subtask. Tài liệu bổ sung cho mỗi bài ba mức: Subtask 1 (30%) là ràng buộc nhỏ để cách ngây thơ lấy điểm, Subtask 2 (30%) và Subtask 3 (40%) là ràng buộc gốc hoặc ràng buộc mở rộng (ví dụ tới 10^{18} thay vì 10^{7}) để thấy rõ vai trò của từng định lí.

Cuối mỗi bài là bảng so sánh: mỗi tầng code chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm. Đầu mỗi bài ghi giới hạn thời gian gốc của Codeforces; riêng bảng so sánh dùng chung một giới hạn 1 giây cho mọi bài để dễ so sánh. Một tầng chỉ được điểm của subtask nếu cho kết quả đúng trong không quá 1 giây, dùng không quá 512 MB bộ nhớ và 8 MB ngăn xếp (ô "sai" là chương trình cho kết quả sai, ô "lỗi" là chương trình bị dừng vì tràn bộ nhớ hoặc tràn ngăn xếp). Thời gian đo bằng g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả đọc và ghi dữ liệu, lấy lần nhanh hơn trong hai lần chạy.

Sử dụng tài liệu

Với mỗi bài, hãy tự suy nghĩ ít nhất 20–30 phút trước khi đọc hướng dẫn. Khi đọc, dừng lại ở câu hỏi then chốt cuối phần **Bế tắc** và tự trả lời trước khi xem phần **Phá bế tắc**.

Mọi code trong tài liệu đã được biên dịch, chạy đúng mọi ví dụ của đề, và đối chiếu với lời giải tham chiếu viết riêng bằng Python trên 60–300 test ngẫu nhiên. Code được viết theo phong cách thi đấu, không chú thích, tên biến đầy đủ, dễ đọc.

Chương 1. Truy hồi tuyến tính bậc thấp

Trọng tâm chương

Truy hồi tuyến tính: mỗi số hạng là tổ hợp tuyến tính cố định của vài số hạng trước. Ghép các số hạng gần nhất thành một vectơ, một bước truy hồi trở thành phép nhân với một ma trận nhỏ, và n bước là ma trận mũ n. Chương này làm quen với ý tưởng đó trên các truy hồi bậc 2, cùng hai lối tắt: chu kì và công thức đóng.

Kiến thức cần nhớ

- Vectơ trạng thái. $(f_i, f_{i-1}) \to (f_{i+1}, f_i)$ là nhân với ma trận 2×2.
- Lũy thừa nhanh. Phép nhân ma trận kết hợp: T^n tính bằng bình phương liên tiếp, $O(k^3 \log n)$.
- Chu kì. In vài số hạng: dãy có thể tuần hoàn ($T^p = I$).
- Gộp đối xứng. Các trạng thái có vai trò như nhau gộp thành một.
- Số âm. Đưa về $[0, M)$ bằng cách cộng M trước khi lấy dư.

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Kĩ thuật chính
1	Jzzhu và các dãy số	450B	Ma trận 2×2, chu kì 6
2	Cây trồng	185A	Ma trận [[3, 1], [1, 3]]
3	Tứ diện	166E	Gộp đỉnh đối xứng

Lê Thành Nam

Bài 1. Jzzhu và các dãy số

Codeforces 450B – Jzzhu and Sequences Thời gian: 1 giây · Bộ nhớ: 256 MB
Đề gốc: https://codeforces.com/problemset/problem/450/B

Đề bài

Dãy f thỏa mãn $f_1 = x$, $f_2 = y$ và $f_i = f_{i-1} - f_{i-2}$ với mọi $i \geq 3$. Cho x, y và n, hãy tính f_n theo modulo $10^9 + 7$ (kết quả là số không âm).

Dữ liệu vào

Dòng đầu chứa x và y.

Dòng thứ hai chứa n.

Ràng buộc chung: $|x|, |y| \leq 10^9$; $1 \leq n$.

Kết quả

Một số nguyên là f_n theo modulo $10^9 + 7$.

Ví dụ

Dữ liệu vào	Kết quả
2 3 3	1
0 -1 2	1000000006

Giải thích ví dụ

Ví dụ 1: $f_3 = f_2 - f_1 = 3 - 2 = 1$. Ví dụ 2: $f_2 = -1 \equiv 10^9 + 6$.

Subtask

Subtask	Ràng buộc	Điểm
Subtask 1	$n \leq 10^6$	30
Subtask 2	$n \leq 10^8$	30
Subtask 3	$n \leq 2 \cdot 10^9$ (ràng buộc gốc)	40

Lê Thành Nam

Hướng dẫn giải

NGÂY THƠ Tính lần lượt

Giữ hai số hạng cuối và tính tiếp theo công thức, nhớ đưa số âm về $[0, 10^9 + 7)$ bằng cách cộng thêm MOD.

Code. Tầng 1. Tính lần lượt, O(n)

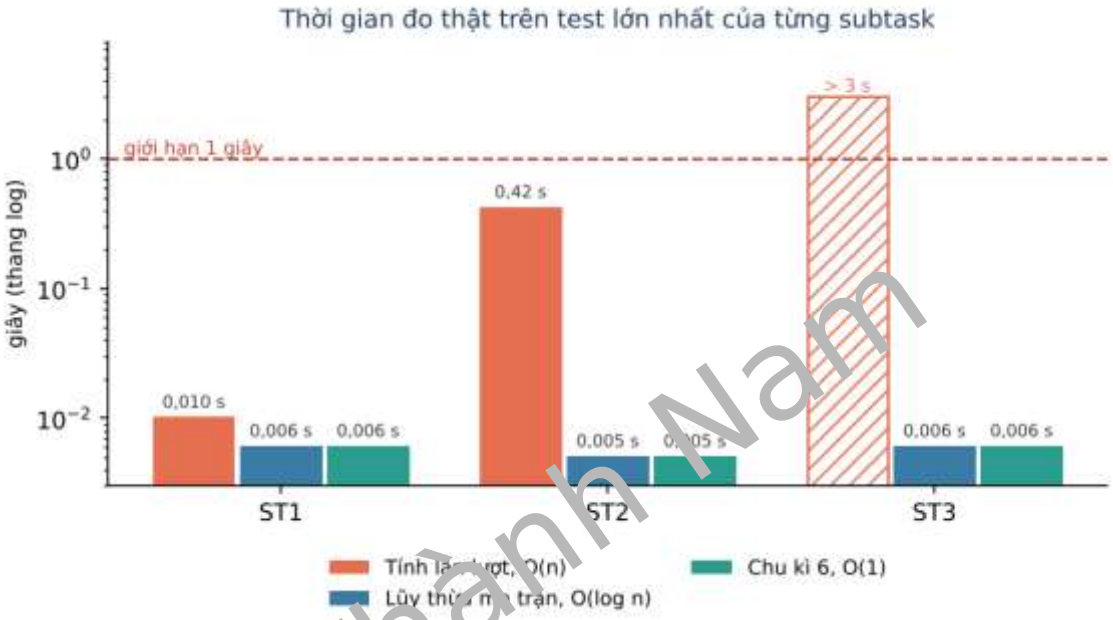
```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
int main() {
    long long x, y, n;
    scanf("%lld %lld %lld", &x, &y, &n);
    long long previous = (x % MOD + MOD) % MOD, current = (y % MOD + MOD) % MOD;
    if (n == 1) { printf("%lld\n", previous); return 0; }
    for (long long i = 3; i <= n; i++) {
        long long next = (current - previous + MOD) % MOD;
        previous = current;
```

```
        current = next;
    }
    printf("%lld\n", current);
}
```

Độ phức tạp: O(n).

BẾ TẮC n tới $2 \cdot 10^9$

$n = 10^8$ chạy 0,42 giây; $2 \cdot 10^9$ thì không kịp (Hình 1.1). Câu hỏi then chốt: Viết thêm vài số hạng của dãy theo x và y. Có gì lặp lại không? Và nếu không lặp, có cách nào "nhảy" nhiều bước một lúc?



Hình 1.1 Thời gian đo thật của ba tầng lời giải.

PHÁ BẾ TẮC Phá bế tắc: Ma trận chuyển trạng thái

Cách tổng quát cho mọi truy hồi tuyến tính: Ghép hai số hạng liên tiếp thành vectơ (f_i, f_{i-1}). Khi đó $(f_{i+1}, f_i) = (f_i - f_{i-1}, f_i)$, tức là nhân với ma trận $T = [[1, -1], [1, 0]]$. Đi n - 2 bước là nhân với T^{n-2}.

Phép nhân ma trận có tính kết hợp, nên T^n tính được bằng lũy thừa nhanh như với số: Bình phương liên tiếp, nhân vào kết quả khi bit của n bằng 1. Chỉ cần khoảng $2 \log_2 n$ phép nhân ma trận 2×2.

Code. Tầng 2. Lũy thừa ma trận, O(log n)

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007;
typedef vector<vector<long long>> Matrix;
Matrix multiply(const Matrix &left, const Matrix &right) {
    int rows = left.size(), middle = right.size(), columns = right[0].size();
    Matrix result(rows, vector<long long>(columns, 0));
    for (int i = 0; i < rows; i++)
        for (int k = 0; k < middle; k++) {
            if (left[i][k] == 0) continue;
            for (int j = 0; j < columns; j++) result[i][j] = (result[i][j] +
left[i][k] * right[k][j]) % MOD;
        }
    return result;
}
Matrix matrixPower(Matrix base, long long exponent) {
    int size = base.size();
    Matrix result(size, vector<long long>(size, 0));
    for (int i = 0; i < size; i++) result[i][i] = 1;
    while (exponent > 0) {
        if (exponent & 1) result = multiply(result, base);
```

Lê Thành Nam